

Funções Recursivas

Objetivos desta unidade são os seguintes:

- Entender que alguns problemas muito complexos podem ter uma solução recursiva simples.
- Aprender a formular programas de forma recursiva.
- Entender e aplicar as três leis da recursão.
- Entender a recursão como uma forma de iteração.
- Implementar a formulação recursiva de um problema.
- Entender como a recursão é implementada por um sistema computacional.

O que é recursão?

Recursão é um método de resolução de problemas que envolve quebrar um problema em subproblemas menores e menores até chegar a um problema pequeno o suficiente para que ele possa ser resolvido trivialmente.

Normalmente recursão envolve uma função que chama a si mesma. Embora possa não parecer muito, a recursão nos permite escrever soluções elegantes para problemas que, de outra forma, podem ser muito difíceis de programar.

O que é recursão?

No contexto da computação, as funções recursivas são aquelas que apresentam chamadas, diretas ou indiretas, a si mesmas.

Uma função recursiva é uma função que chama a si mesma. Para evitar que uma função se repita indefinidamente, ela deve conter pelo menos uma instrução de seleção. Essa instrução examina uma condição chamada de caso-base para determinar se deve parar ou continuar com uma etapa recursiva.

Funções Recursivas

- Recursividade é uma forma interessante de resolver problemas. Essa forma se aplica quando um problema pode ser dividido em problemas menores (subproblemas) de mesma natureza que o problema original.
- Como os subproblemas têm a mesma natureza do problema original, o mesmo método usado para reduzir o problema pode ser usado para reduzir os subproblemas.
- Mas, até quando devemos dividir o problema em subproblemas? Quando parar de reduzir? Quando o subproblema obtido for um caso trivial, para o qual se conhece a solução.
- Se o método usado para reduzir o problema for implementado como uma função, usar o mesmo método é chamar a função dentro da própria função

Aqui está um exemplo que ilustra o que queremos dizer com uma função que chama a si mesma:

```
def countdown(n):  
    print(n)  
    countdown(n-1)
```

Uma função recursiva para imprimir na vertical um numero inteiro.

Exemplo 3214

Exibir

3

2

1

4

Funções Recursivas

Exemplo: Calcular 10!

$$10! = 10 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4 \times 3 \times 2 \times 1$$

- Então: $10! = 10 \times 9!$
- O problema de calcular 10! reduziu-se ao problema de calcular 9!
- Mas $9! = 9 \times 8!$
- Até quando devemos reduzir o problema? Qual é o caso trivial de cálculo do fatorial? É a definição de $0! = 1$.
- Então podemos escrever:

$$n! = \begin{cases} 1 & \text{se } n = 0 \\ n \times (n-1)! & \text{se } n > 0 \end{cases}$$

$$n! = \begin{cases} 1 & \text{se } n = 0 \\ n \times (n-1)! & \text{se } n > 0 \end{cases}$$

```
def fat(n):  
    if n == 0:  
        return 1  
    else:  
        return n*fat(n-1)
```

```
n = int(input('informe um número'))  
print(fat(n))
```

OBSERVE que a função fat chama a si mesma

Como funciona essa função fat? A função usa um mecanismo conhecido como pilha de execução!

Numa pilha de execução, os resultados parciais são empilhados e são desempilhados somente quando é possível realizar o cálculo do resultado (ou quando se chega ao caso trivial).

```
def fat(n):  
    if n == 0:  
        return 1  
    else:  
        return n*fat(n-1)
```

$$n! = \begin{cases} 1 & \text{se } n = 0 \\ n \times (n-1)! & \text{se } n > 0 \end{cases}$$

Pilha de Execução

<code>fat(0)</code>
<code>fat(1)</code>
<code>fat(2)</code>
<code>fat(3)</code>
<code>fat(4)</code>

return 1 (caso trivial)

return 1*fat(0)

return 2*fat(1)

return 3*fat(2)

return 4*fat(3)

Resultados

1
1*1 = 1
2*1 = 2
3*2 = 6
4*6 = 24

As três leis da recursão

Todos os algoritmos recursivos devem obedecer a três leis importantes:

1. Um algoritmo recursivo deve ter um caso básico
2. Um algoritmo recursivo deve mudar o seu estado e se aproximar do caso básico.
3. Um algoritmo recursivo deve chamar a si mesmo, recursivamente.

Implementação de Algoritmos Dividir para Conquistar

Alguns dos algoritmos mais eficientes da computação dividem um problema grande em subproblemas idênticos menores.

Ordenação: QuickSort e MergeSort usam recursão para ordenar grandes volumes de dados muito mais rápido do que métodos iterativos simples ($O(n \log n)$ versus $O(n^2)$).

Busca: Busca binária em estruturas encadeadas/árvores.

- Referência: <https://panda.ime.usp.br/pensepy/static/pensepy/12-Recursao/recursionsimple-ptbr.html#lst-recsum>
- https://edisciplinas.usp.br/pluginfile.php/4121279/mod_resource/content/1/SolucaoLista.pdf
- <https://www.youtube.com/watch?si=YU4XNhnZJ7kw3Sh3&v=M7c-m2xN9FQ&feature=youtu.be>